

1) ある会社は、AWS CodeCommit でアプリケーションのソースコードを管理しています。同社は AWS CodePipeline を使用してアプリケーションの CI/CD パイプラインの作成をしています。CodeCommit リポジトリのメインブランチが変更されたら、自動的にパイプラインが開始される必要があります。変更は毎日頻繁に発生するため、パイプラインの応答性はできる限り高くしなければなりません。

これらの要件を満たすために DevOps エンジニアは何をすべきですか。

- A) リポジトリのメインブランチに変更がないか定期的にチェックするようにパイプラインを設定する。変更が検出されたらパイプラインを開始する。
- B) リポジトリのメインブランチの変更を検出するように Amazon EventBridge (Amazon CloudWatch Events) ルールを設定する。変更に応じて開始するようにパイプラインを設定する。
- C) AWS Lambda 関数を定期的に行うようにリポジトリを設定する。リポジトリのメインブランチをチェックし、変更が検出されたらパイプラインを開始するように関数を設定する。
- D) リポジトリのメインブランチが変更されたら Amazon Simple Notification Service (Amazon SNS) トピックに通知を発行するようにリポジトリを設定する。パイプラインを SNS トピックに登録する。

2) ある DevOps チームには、既存の Amazon S3 バケットに会社の重要なアセットを保存するアプリケーションがあります。このチームは単一の AWS リージョンを使用しています。会社の新しいポリシーに従い、アプリケーションを複数のリージョンにデプロイすることになりました。アセットには常にアクセスできなければなりません。ユーザーは、アセットにアクセスするために 同じエンドポイントを使用する必要があります。

最も運用効率の高い方法でこれらの要件を満たすために、チームが実行すべきステップの組み合わせはどれですか。(3 つ選択)

- A) AWS CloudFormation StackSets を使用して、必要な各リージョンに新しい S3 バケットを 1 つずつ作成し、バケットのバージョンングを有効にする。既存の S3 バケットから新しい S3 バケットにアセットをコピーする。AWS Lambda 関数を作成して、プライマリリージョンの新しい S3 バケットに追加されたファイルを他のリージョンにコピーする。
- B) AWS CloudFormation StackSets を使用して、必要な各リージョンに新しい S3 バケットを 1 つずつ作成し、バケットのバージョンングを有効にする。プライマリリージョンの新しい S3 バケットで、複数の S3 レプリケーションルールを作成してすべてのコンテンツを追加のリージョンにレプリケートする。既存の S3 バケットのアセットをプライマリリージョンの新しい S3 バケットにコピーする。
- C) Amazon CloudFront ディストリビューションを作成する。各 S3 バケットに新しいオリジンを設定する。新しく作成したすべてのオリジンを含むオリジングループを作成する。新しいオリジングループを使用するように、そのディストリビューションのデフォルトの動作を更新する。
- D) Amazon CloudFront ディストリビューションを作成する。各 S3 バケットに新しいオリジンを設定する。そのオリジンの可用性を検証し、使用可能な近くのオリジンにビューワーリクエストを送信する Lambda@Edge 関数を作成する。
- E) Amazon Route 53 エイリアスレコードを作成する。新しく作成された S3 バケットをターゲットとして使用するフェイルオーバールーティングポリシーを設定する。
- F) Amazon Route 53 エイリアスレコードを作成する。Amazon CloudFront ディストリビューションをターゲットとして使用するシンプルルーティングポリシーを設定する。

3) ある会社は、AWS CodeBuild を使用してアプリケーションを構築しています。会社のポリシーに従い、すべてのビルドアーティファクトを保管時に暗号化する必要があります。この会社では、アーティファクトへのアクセスを、運用 IAM ロールを引き受けるアクセス許可を持つ運用 IAM グループの IAM ユーザーに制限する必要があります。

これらの要件を満たすソリューションはどれですか。

- A) CodeBuild のビルド仕様にビルド後、コマンドを追加してビルドオブジェクトを Amazon S3 バケットにプッシュする。リクエストに x-amz-server-side-encryption ヘッダーが含まれていない場合はバケットにアップロードできないように、バケットポリシーを設定する。運用 IAM グループを参照する NotPrincipal 要素を持つすべてのアクションに対する Deny ステートメントを追加する。
- B) CodeBuild のビルド仕様に、ビルド後コマンドを追加してビルドオブジェクトを Amazon S3 バケットにプッシュする。AWS Lambda 関数を呼び出してオブジェクトを取得、暗号化し、タグキーが Encrypted でタグ値が True の S3 バケットにそのオブジェクトを戻すように S3 イベント通知を設定する。運用 IAM グループを参照する NotPrincipal 要素を持つすべてのアクションに対する Deny ステートメントにバケットポリシーを設定する。そのポリシーに、Encrypted タグを参照する Condition 要素を含める。
- C) CodeBuild のビルド仕様に、ビルド後コマンドを追加して S3 のデフォルトの暗号化が有効な Amazon S3 バケットにビルドオブジェクトをプッシュする。運用 IAM ロールを参照する NotPrincipal 要素を持つすべてのアクションに対する Deny ステートメントを含むバケットポリシーを設定する。
- D) CodeBuild のビルド仕様に、ビルド後コマンドを追加して、AWS Key Management Service (AWS KMS) Encrypt API オペレーションを呼び出し、指定された KMS キーに暗号化するために AWS KMS にアーティファクトを渡す。暗号化されたアーティファクトを Amazon S3 バケットにプッシュする。運用 IAM グループを、指定された KMS キーの唯一のユーザーとして設定する。

4) ある DevOps エンジニアは、AWS でアプリケーションのためのブルー/グリーンデプロイプロセスを実装する必要があります。DevOps エンジニアは、環境間でトラフィックを徐々にシフトしなければなりません。

アプリケーションは、Application Load Balancer (ALB) の背後にある Amazon EC2 インスタンスで実行されます。このインスタンスは、Amazon EC2 Auto Scaling グループで実行されます。このアプリケーションは、Amazon RDS Multi-AZ DB インスタンスにデータを保存します。Amazon Route 53 は外部 DNS を提供します。

これらの要件を満たすために、DevOps エンジニアが取るべきステップの組み合わせはどれですか。(3 つ選択)

- A) 同じ ALB の背後に 2 つ目の Auto Scaling グループを作成する。
- B) 2 つ目の ALB の背後に 2 つ目の Auto Scaling グループを作成する。
- C) Route 53 で、新しい環境を指す 2 つ目のエイリアスレコードを作成する。フェイルオーバールーティングポリシーを使用して 2 つのレコードのいずれかを選択する。
- D) Route 53 で、新しい環境を指す 2 つ目のエイリアスレコードを作成する。加重ルーティングポリシーを使用して 2 つのレコードのいずれかを選択する。
- E) プライマリ RDS DB インスタンスを使用するように新しい EC2 インスタンスを設定する。
- F) スタンバイ RDS DB インスタンスを使用するように新しい EC2 インスタンスを設定する。

5) ある会社は、最新バージョンの Amazon Linux 2 AMI を使用する Amazon EC2 インスタンスでアプリケーションを実行しています。サーバー管理者が新しいセキュリティパッチを適用するときは、サーバー管理者は影響を受けるインスタンスを手動でサービスから削除し、インスタンスにパッチを適用した後、インスタンスをサービスに戻します。

新しいセキュリティポリシーに従い、セキュリティパッチをリリース後 7 日以内に適用する必要があります。この会社のセキュリティチームは、すべての EC2 インスタンスがこのポリシーに準拠していることを検証しなければなりません。パッチ適用は、ユーザーへの影響が最も少ない時間帯に実施する必要があります。

これらの要件へのコンプライアンスを自動化するソリューションはどれですか。

- A) SSH 経由ですべてのインスタンスにパッチをダウンロードして適用するように AWS CodeBuild プロジェクトを設定する。Amazon EventBridge (Amazon CloudWatch Events) スケジュールのルールを使用して、メンテナンスウィンドウ中に CodeBuild プロジェクトを実行する。
- B) AWS Systems Manager パッチマネージャーを使用してパッチベースラインを作成する。EC2 インスタンスで、AWS CLI を使用してパッチマネージャーから最新のパッチを取得するスクリプトを作成する。cron ジョブを作成して、メンテナンスウィンドウ中にそのスクリプトを実行するようにスケジュールする。
- C) 利用可能なセキュリティパッチを適用するスクリプトを作成する。cron ジョブを作成して、メンテナンスウィンドウ中にそのスクリプトを実行するようにスケジュールする。そのスクリプトと cron ジョブをアプリケーション AMI にインストールする。アプリケーションを再デプロイする。
- D) AWS Systems Manager パッチマネージャーパッチグループにすべての EC2 インスタンスを追加する。パッチマネージャーを使用してパッチベースラインを作成する。パッチベースラインを適用するためのメンテナンスウィンドウを設定する。

6) ある会社は、すべての AWS アカウントで AWS CloudTrail を使用し、一元化された Amazon S3 バケットにすべての追跡を送信しています。S3 イベント通知と AWS Lambda 関数を使用して、指定されたイベントをサードパーティーのロギングツールに送信します。

この会社では、セキュリティオペレーションセンターを設置するためにセキュリティサービスプロバイダーを採用しました。このセキュリティサービスプロバイダーは、Amazon Simple Queue Service (Amazon SQS) キューを介して CloudTrail ログを受信したいと考えています。

S3 イベント通知と Lambda 関数を使用して、引き続きサードパーティーのロギングツールにイベントを送信する必要があります。

これらの要件を満たす最も運用効率の高い方法はどれですか。

- A) すべての CreateObject イベントに関する通知を S3 バケットに追加し、すべてのオブジェクトを SQS キューに送信する。
- B) 既存の S3 イベント通知の送信先を Amazon Simple Notification Service (Amazon SNS) トピックに置き換える。そのトピックに Lambda 関数と SQS キューを登録する。
- C) 既存の S3 イベント通知の送信先を Amazon Kinesis データストリームに置き換える。Lambda 関数と SQS キュー用のコンシューマーを作成する。
- D) ログを Amazon CloudWatch Logs に送信するように証跡を設定する。SQS キューを CloudWatch Logs ロググループに登録する。

7) ある会社は、AWS アカウントのセキュリティポリシーの見直しを行っています。さまざまな国にスタッフメンバーがいるため、IAM ID に関連する異常な動作がないか AWS アカウントをモニタリングしたいと考えています。

異常なアクティビティが検出されたスタッフメンバーには通知を送信する必要があります。また、そのユーザーのチームリーダーにも通知を送信する必要があります。通知は外部のメッセージングプラットフォームから送信します。このプラットフォームでは、受信者ごとにターゲットユーザー ID が必要です。

同社は既に、AWS に API を持っていて、IAM ユーザー名からスタッフメンバーとチームリーダーのユーザー ID を返すために使用できます。AWS アカウントの管理には AWS Organizations を使用しています。

これらの要件を満たすソリューションはどれですか。

- A) 組織内の 1 つのアカウントを Amazon GuardDuty 管理者に指定する。この会社の AWS アカウントを、その GuardDuty 管理者アカウントに関連付けた GuardDuty メンバーアカウントとして追加する。ユーザー ID を検索し、外部メッセージングプラットフォームに通知を送信する AWS Lambda 関数を作成する。GuardDuty 管理者アカウントに、Impact:IAMUser/AnomalousBehavior 通知タイプを照合して Lambda 関数を呼び出す Amazon EventBridge (Amazon CloudWatch Events) ルールを作成する。
- B) 組織内の 1 つのアカウントを Amazon Detective 管理者に指定する。この会社の AWS アカウントを、その Detective 管理者アカウントに関連付けた Detective メンバーアカウントとして追加する。ユーザー ID を検索し、外部メッセージングプラットフォームに通知を送信する AWS Lambda 関数を作成する。Detective 管理者アカウントに、Impact:IAMUser/AnomalousBehavior 通知タイプを照合して Lambda 関数を呼び出す Amazon EventBridge (Amazon CloudWatch Events) ルールを作成する。
- C) 組織内の 1 つのアカウントを Amazon GuardDuty 管理者に指定する。この会社の AWS アカウントを、その GuardDuty 管理者アカウントに関連付けた GuardDuty メンバーアカウントとして追加する。ユーザー ID を検索し、外部メッセージングプラットフォームに通知を送信する AWS Lambda 関数を作成する。GuardDuty 管理者アカウントに、Impact:IAMUser/AnomalousBehavior 通知タイプを照合して Lambda 関数を呼び出す Amazon Simple Notification Service (Amazon SNS) トピックを作成する。
- D) 組織内の 1 つのアカウントを Amazon Detective 管理者に指定する。この会社の AWS アカウントを、その Detective 管理者アカウントに関連付けた Detective メンバーアカウントとして追加する。ユーザー ID を検索し、外部メッセージングプラットフォームに通知を送信する AWS Lambda 関数を作成する。Detective 管理者アカウントに、Impact:IAMUser/AnomalousBehavior 通知タイプを照合して Lambda 関数を呼び出す Amazon Simple Notification Service (Amazon SNS) トピックを作成する。

8) ある開発チームは、3 つの AWS リージョンにまたがる大規模なお客様ベースがあるアプリケーションを設計しています。このアプリケーションで使用する Amazon DynamoDB テーブルは、低レイテンシーのデータアクセスを実現するために 3 つすべてのリージョンで利用できる必要があります。あるリージョンでテーブルが更新されたら、その変更を他のリージョンにシームレスに伝播しなければなりません。

最小限の運用コストでこれらの要件を満たすために、DevOps エンジニアはどのようにテーブルを設定すべきですか。

- A) 3 つのリージョンそれぞれに DynamoDB テーブルを作成する。各テーブルに同じ名前を付ける。
- B) 3 つのリージョンそれぞれに 3 つの DynamoDB テーブルを設定する。DynamoDB 用 AWS SDK を使用してテーブル間でデータ変更を同期する。
- C) 3 つのリージョンを含む、マルチリージョンおよびマルチアクティブの DynamoDB グローバルテーブルを設定する。
- D) DynamoDB グローバルテーブルを使用して 1 つのリージョンにプライマリテーブルを設定し、他のリージョンそれぞれにリードレプリカを設定する。

9) ある会社には、パブリック Application Load Balancer (ALB) の背後にある Amazon EC2 インスタンスのフリートで実行されているレガシー API があります。ALB ではアクセスのログ記録が有効になっており、そのアクセスログは Amazon S3 に保存されます。api.example.com というホスト名でその API を使用できます。ホスト名の管理には Amazon Route 53 を使用しています。

開発者は、API エンドポイントごとに異なる AWS Lambda 関数を使用することで、5 つの API エンドポイントを再構築しました。DevOps エンジニアは、その Lambda 関数の新しいバージョンを一部のお客様でランダムにテストしたいと考えています。既存のログ処理サービスとの互換性を確保するために、ALB のアクセスログに影響を与えないようにテストする必要があります。

DevOps エンジニアはこれらの要件を満たすためにこのテストをどのように実行すべきですか。

- A) EC2 インスタンスの既存のターゲットグループに、5 つの Lambda 関数をターゲットとして追加する。各 Lambda 関数ターゲットのターゲットグループの重みを、EC2 インスタンスターゲットより小さくなるように設定する。ALB のデフォルトルールを修正し、ターゲットグループレベルの維持設定を有効にする。
- B) すべての Lambda 関数を個別のターゲットとして含む、単一のターゲットグループを作成する。ALB に、API エンドポイントのホスト名に一致するホストヘッダー条件を含む新しいリスナールールを作成する。リスナールールにターゲットグループを追加する。新しいターゲットグループの重みを、デフォルトルールのターゲットグループより小さくなるように指定する。
- C) Lambda 関数ごとに新しい ALB とターゲットグループを作成する。各エンドポイントに一致したらトラフィックをターゲットグループに転送するホストヘッダー条件を含む新しいリスナールールを作成する。重みが 10 の新しい Route 53 エイリアスレコードを作成する。ホスト名 api.example.com の既存の Route 53 レコードの重みを 90 に更新する。
- D) Lambda 関数ごとに新しいターゲットグループを作成する。ALB に、さまざまな各エンドポイントに一致するパス条件を含む新しいリスナールールを作成する。そのエンドポイントの Lambda 関数ターゲットグループとインスタンススペースのターゲットグループ間で重み付けするようにルールを設定する。

10) ある DevOps エンジニアは、AWS でレガシーアプリケーションを管理しています。そのアプリケーションは、単一の Amazon EC2 インスタンスで実行されるモノリシックな Windows プログラムです。アプリケーションのソースコードを入手できないため、アプリケーションを修正することはできません。

アプリケーションにメモリリークがあり、EC2 インスタンスでメモリ使用率が 90% を超えると誤動作します。DevOps エンジニアは、EC2 インスタンス上に統合された Amazon CloudWatch エージェントを設定し、オペレーションシステムのメモリ使用率メトリクスを収集しています。

DevOps エンジニアは、アプリケーションの誤動作を防ぐソリューションを実装する必要があります。

最も運用効率の高い方法でこれらの要件を満たすステップの組み合わせはどれですか。(2 つ選択)

- A) メモリ使用率が 80% を超えると Amazon Simple Notification Service (Amazon SNS) トピックに発行する Amazon EventBridge (Amazon CloudWatch Events) ルールを作成する。
- B) Amazon CloudWatch Logs でメモリ使用率に対するメトリクスフィルターを作成する。メモリ使用率フィルターに CloudWatch アラームを作成する。メモリ使用率が 80% を超えると Amazon Simple Notification Service (Amazon SNS) トピックに発行するアラームを設定する。
- C) メモリ使用率メトリクスに CloudWatch アラームを作成する。メモリ使用率が 80% を超えると Amazon Simple Notification Service (Amazon SNS) トピックに発行するアラームを設定する。
- D) AWS Systems Manager Run Command を使用して、アプリケーションを再起動する AWS Lambda 関数を設定する。Amazon Simple Notification Service (Amazon SNS) トピックに Lambda 関数を登録する。
- E) アプリケーションを再起動するスクリプトを実行するように EC2 インスタンスを設定する。Amazon Simple Notification Service (Amazon SNS) トピックに EC2 インスタンスを登録する。

解答

1) B - オプション B が[推奨されるソリューション](#)であり、与えられたオプションの中で応答性が最も高くなります。変更によってイベントが直接生成され、そのイベントによってパイプラインが直接開始されます。オプション A の定期チェックは機能しますが、次の定期チェックが実行されるまでパイプラインは起動しません。オプション C の機能は、AWS CodeCommit でサポートされていません。オプション D は、パイプラインの開始方法として有効ではありません。

2) B、C、F - この問題は 3 つのパートに分けられます。パート 1 は、ワークロードを複数の AWS リージョンにデプロイする方法です。パート 2 は、複数のデプロイに対して単一のエンドポイントからアクセスを提供する方法です。パート 3 は、DNS を使用してフェイルオーバーイベントを処理する方法です。

パート 1: オプション B の [AWS CloudFormation StackSets](#) は、リージョン固有の Amazon S3 バケットに運用効率の高いマルチリージョンデプロイ戦略を提供します。[S3 レプリケーション](#)により、プライマリリージョンにある新規オブジェクトと既存のオブジェクトが[複数のデプロイリージョン](#)にコピーされます。運用効率が劣るため、オプション A は正しくありません。S3 レプリケーションはカスタムコードなしで適切な機能を提供できるため、AWS Lambda 関数は不要です。

パート 2: オプション C の Amazon CloudFront ディストリビューションを使用すると、単一のエンドポイントが複数のオリジンに解決されます。CloudFront のカスタムオリジンを設定すると、接続タイムアウトの短縮、接続試行回数の削減、またはその両方を必要とする[高可用性オリジンフェイルオーバー](#)を構築できます。オプション D は実行可能ですが、Lambda@Edge 関数内にカスタムコードが含まれるため、運用効率が劣ります。オリジン設定内でネイティブに処理されるため、カスタムコードは不要です。

パート 3: オプション F では、CloudFront オリジン設定によってフェイルオーバーが処理されるため、Route 53 が CloudFront ディストリビューションに[シンプルルーティングポリシー](#)のユーザーフレンドリーなドメイン名を提供します。フェイルオーバールーティングの利点が複数のレコードで活かされないため、オプション E は正しくありません。

3) C - [Amazon S3 のデフォルトの暗号化](#)により、アーティファクトが保管時に確実に暗号化されます。Deny ステートメントと、運用 IAM ロールを参照するように設定した [NotPrincipal](#) 要素を使用すると、そのロールを使用するリクエスト以外は、S3 バケットへのアクセスが拒否されます。このシナリオは、運用ロールにそのバケットにアクセスできる許可ポリシーがあることを意味しています。

バケットポリシーが IAM ロールではなく IAM グループを参照しているため、オプション A と B は正しくありません。オプション A は、AWS がバケットポリシーによる暗号化の実施よりも[デフォルトの暗号化の使用を推奨している](#)という理由からも正しくありません。また、オプション B では、アーティファクトが一時的に暗号化されずに保存されています。AWS Key Management Service (AWS KMS) [Encrypt API オペレーション](#)は、パスワードなどのプレーンテキスト値の暗号化には有効ですが、ビルドアーティファクトファイル、アーカイブ、オブジェクトの暗号化には有効ではないため、オプション D は正しくありません。

4) B、D、E - [ブルー/グリーンデプロイ](#)は、2 つの個別の環境で構成されます。ブルー環境には、現在の本番バージョンのアプリケーションを実行している Auto Scaling グループの Amazon EC2 インスタンスがあります。グリーン環境には、新しいバージョンのアプリケーションを実行するもう 1 つの Auto Scaling グループの EC2 インスタンスがあります。各 Auto Scaling グループはそれぞれの Application Load Balancer (ALB) の背後にあるため、Amazon Route 53 で 2 つのエイリアスレコードをエンドポイントとして設定し、[加重ルーティングポリシー](#)を使用して、ブルー環境用の ALB からグリーン環境用の ALB にトラフィックを徐々にシフトできます。新しいリリースのためにスキーマの変更が必要な場合を除き、カットオーバー中のデータの一貫性を保つためには、両方の環境が同じデータベースを指すようにすることが推奨されます。

Route 53 を使用してトラフィックを徐々にシフトできるようにするには、エンドポイントとして 2 つの ALB が必要になるため、オプション A は正しくありません。オプション C が正しくないのは、ヘルスチェックで障害が検出されない限り、フェイルオーバールーティングポリシーによってすべてのトラフィックが単一のエンドポイントに送信されてしまうためです。これにより、このオプションではトラフィックを徐々にシフトすることができません。Amazon RDS Multi-AZ DB インスタンスのスタンバイインスタンスはホットスタンバイであり、読み取りや書き込みには利用できないため、オプション F は正しくありません。

5) D - AWS Systems Manager の機能である[パッチマネージャー](#)を使用すると、[パッチベースライン](#)で定義した承認済みパッチのリストに従って、メンテナンスウィンドウ中にセキュリティパッチが自動的に適用されます。この会社のセキュリティチームは、Systems Manager コンソールでインスタンスの[パッチコンプライアンス](#)を表示したり、AWS CLI を使用して要約を取得したりすることができます。

AWS CodeBuild はソースコードをアーティファクトにビルドするため、オプション A は正しくありません。CodeBuild は、インスタンスにパッチをデプロイしません。Amazon Linux 2 にプリインストールされている Systems Manager Agent (SSM Agent) をパッチを取得するようにスケジュールする必要はないため、オプション B は正しくありません。このシナリオで必要なのは、パッチ適用設定を [Systems Manager のメンテナンスウィンドウ](#)に関連付けることだけです。セキュリティチームがパッチコンプライアンスを検証する方法がないため、オプション C は正しくありません。また、このオプションの cron ジョブには単一障害点があります。

6) B - 多くのコンシューマーに 1 つのイベントの [Amazon S3 イベント通知を送信するファンアウトメスセーシングシナリオ](#)を実装するために、[S3 イベント通知](#)の送信先を Amazon Simple Notification Service (Amazon SNS) トピックに変更できます。Lambda 関数コードを変更することなく、[AWS Lambda 関数](#)や [Amazon Simple Queue Service \(Amazon SQS\) キュー](#)などの複数のコンシューマーをそのトピックに登録できます。

オプション A は不可能であり、通知イベントのプレフィックスとサフィックスが重複しているため、["Configuration is ambiguously defined \(構成の定義があいまいです\)"](#)というエラーが発生します。Amazon Kinesis データストリームは [S3 イベント通知の送信先](#)として有効ではないため、オプション C は正しくありません。オプション D は不完全なソリューションであるため、正しくありません。SQS キューを Amazon CloudWatch Logs ロググループに直接登録することはできません。

7) A - [Amazon GuardDuty](#) の結果は、AWS Lambda 関数のターゲットを呼び出すことができるイベントとして [Amazon EventBridge \(Amazon CloudWatch Events\)](#) に発行されます。[Amazon Detective](#) だけでは異常なアクティビティが検出されないため、オプション B は正しくありません。Detective は、特定の結果に関連する分析情報を提供します。[Amazon Simple Notification Service \(Amazon SNS\)](#) では、[メッセージを属性でフィルターできます](#)が、メッセージの内容ではフィルターできないため、オプション C は正しくありません。SNS トピックに発行するには、EventBridge (CloudWatch Events) ルールが必要です。オプション B および C と同じ理由で、オプション D は正しくありません。

8) C - [Amazon DynamoDB グローバルテーブル](#)は、マルチリージョンおよびマルチアクティブのワークロードに使用できる単一リージョンのテーブルとして開始されます。グローバルテーブルを使用すると、リージョン固有のワークロードで低レイテンシーのデータアクセスを実現でき、レプリケーションソリューションの設定や管理は必要ありません。

リージョンごとに個別のテーブルを使用すると、追加のレプリケーションソリューションが必要になるため、オプション A は正しくありません。複数のテーブルにわたる同期処理の開発と管理が余分な運用コストとなるため、オプション B は正しくありません。グローバルテーブルはリードレプリカがないマルチリージョンおよびマルチアクティブのテーブルであるため、オプション D は正しくありません。

9) D - このシナリオは、[ブルー/グリーンデプロイ](#)と Canary デプロイメントに似ています。このソリューションに必要なのは、既存の [Application Load Balancer \(ALB\)](#) だけです。ターゲットグループでは、[登録済みのターゲットとして 1 つの AWS Lambda 関数](#)がサポートされています。そのため、このソリューションでは、エンドポイントごとに 1 つずつ、5 つのターゲットグループが必要です。各エンドポイントには独自のパスがあるため、既存の EC2 ターゲットグループと新しい Lambda 関数ターゲットグループに渡ってリクエストの加重ディストリビューションができるように[リスナールールに新しいパス条件が必要です](#)。

1 つのターゲットグループに複数の Lambda 関数を登録することはできないため、オプション A と B は正しくありません。また、オプション A は、ターゲットグループレベルの維持設定によって、限定的にテストする際の加重ルーティングの利点が損なわれます。オプション B は、加重ルールが個々のルールレベルで割り当てられ、複数のルールにわたって評価されないため正しくありません。新しいロードバランサー ID に基づいて異なるアクセスログが生成されると、ALB アクセスログに影響を与えるため、オプション C は正しくありません。また、ホストヘッダー条件を含むリスナールールは、URI レベルのテストには効果的ではありません。

10) C、D - 統合された Amazon CloudWatch エージェントによって、アラームに直接使用できる [CloudWatch メトリクス](#)としてシステムレベルのメトリクスが発行されるため、オプション C は正解です。[Amazon Simple Notification Service \(Amazon SNS\)](#) トピックに登録されている [AWS Lambda 関数](#)は、[AWS API または SDK](#) を使用して [AWS Systems Manager Run Command](#) を開始することによって Amazon EC2 インスタンスにコマンドを送信できるため、オプション D は正解です。

オプション A と B は正解ほど運用効率が良くないため、正しくありません。オプション A では、CloudWatch アラームが Amazon SNS に直接通知できるため、Amazon EventBridge (Amazon CloudWatch Events) ルールは必要ありません。発行されたシステムレベルの CloudWatch メトリクスがアラームのメトリクスを生成するための CloudWatch Logs ベースのメトリクスフィルターの必要性を無効にするため、オプション B は正しくありません。EC2 インスタンスを SNS トピックに登録できないため、オプション E は正しくありません。